



Spl3 Project Presentation

TestSmellRank: A Test Smell Prioritization Tool

Presented By:
Tanjuma Tabassum
BSSE 1312

Supervised By:
Dr. Md. Shariful Islam
Professor, IIT, DU



INTRODUCTION

Test smells are poor testing practices that make test code hard to maintain and prone to bugs. Tools like PyNose, Pytest-smell can detect them, but developers lack prioritization guidance.

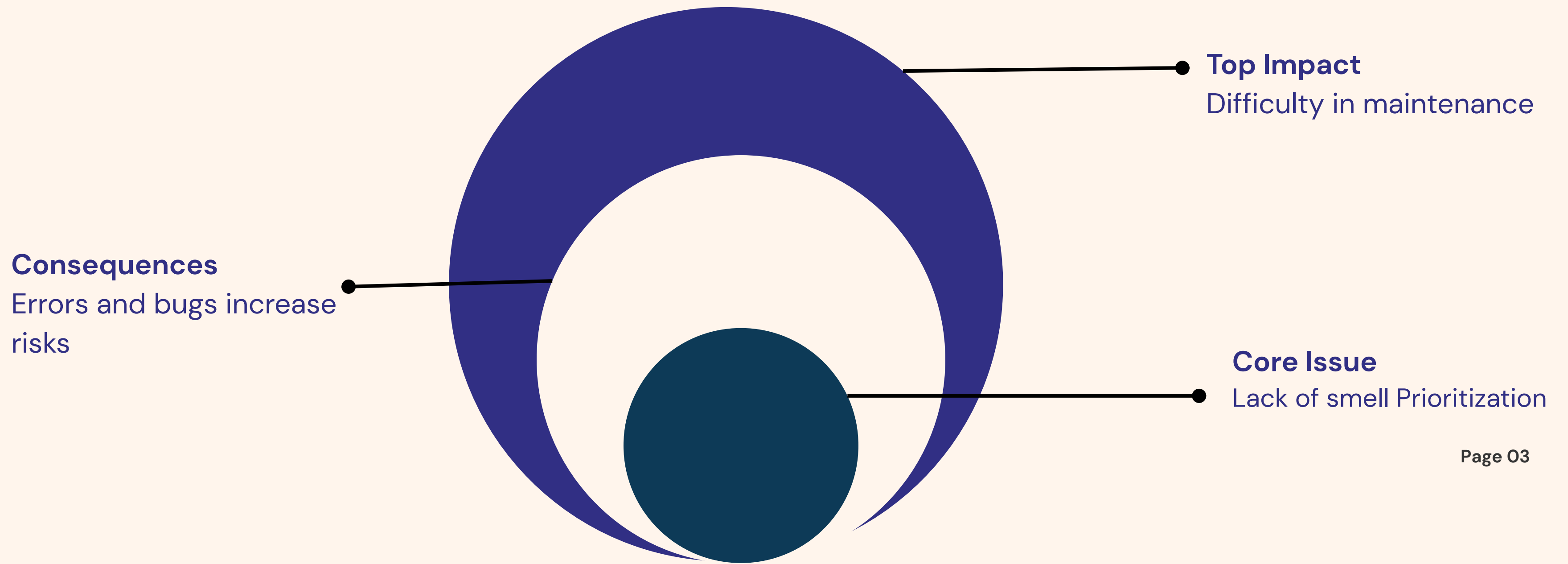
TestSmellRank aims to automatically detect, evaluate and rank test smells in Python project to help developers decide which issues to fix first.

- Automatic Detection using a Custom AST-Based Static Analysis Approach.
- Severity Scoring based on Commit History Metrics.
- Integration of Developer Feedback.
- Generation of a Prioritized Report.
- Compare two Run

MOTIVATION

Why Prioritization is Crucial

Tools like PyNose successfully identify test smells, developers often lack a clear objective metric to determine which issues pose the highest risk to the project's stability and maintainability.



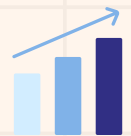
FEATURES



User Authentication



Automatic detection of test smells (Ast-based approach)



Metric-based ranking (CP-Change Proneness, FP-Fault Proneness)



Developer feedback integration



Final prioritized report with severity & reference lines



Comparison of two runs of a specific project

QFD

NORMAL REQUIREMENTS

- Detect multiple test smell types in Python projects
- Compute Change Proneness (CP) and Fault Proneness (FP) from Git commit history
- Accept both GitHub repositories and local ZIP files as input
- Calculate priority scores using a standard CP–FP method
- Provide a simple and intuitive user interface for analysis and results

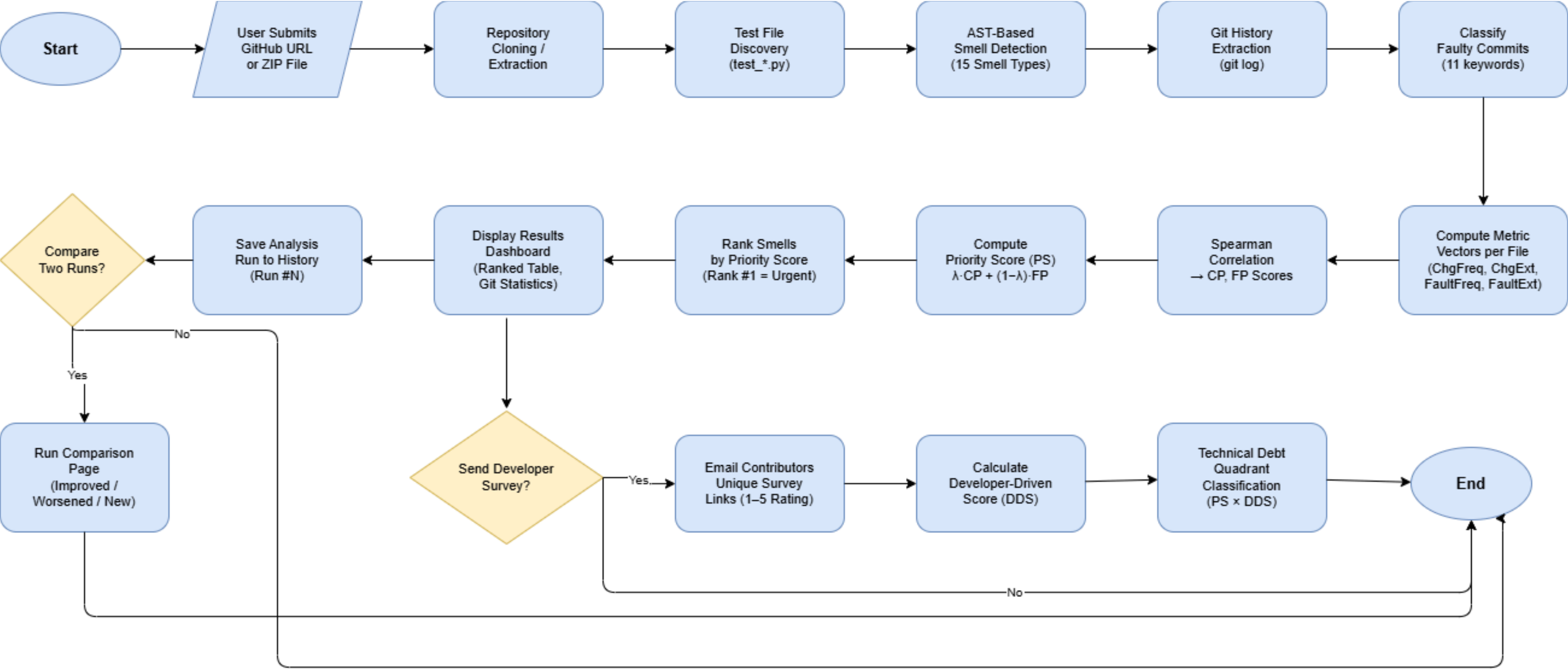
EXPECTED REQUIREMENTS

- Store and retrieve analysis history for comparison
- Generate reports with ranked smells and code locations
- Classify test smells using the Technical Debt Quadrant framework

EXCITING REQUIREMENTS

- Integrate developer surveys to include perception-based prioritization
- Support weighted priority scoring for CP and FP
- Compare 2 runs of a project

WORKFLOW



TOOLS AND TECHNOLOGIES

- Python
- FastAPI (backend)
- ReactJs (frontend)
- MongoDB

PROJECT DEMONSTRATION



TestSmellRank

Login

Register

Detect & Prioritize Test Smells in Your Code

Analyze and rank test smells in your Python projects with our advanced detection system

Upload your project or provide a GitHub repository link to get started



Smart Detection

Automatically detect test smells in Python projects



Detailed Analysis

Get comprehensive reports and rankings



Fast Processing

Quick analysis of your entire test suite

Landing Page

TestSmellRank

Create your account

Full Name

Enter your full name

Email

Enter your email

Password

Enter your password (min 6 characters)

Register

Already have an account? [Sign in here](#)

Registration page

tests\test_assertionless.py		No test smells detected		CLEAN
tests\test_conditional.py	Redundant Assertion	4	Assertion always passes — provides no real verification	2 SMELLS
	Conditional Test Logic	3	Test contains an if/else branch	
tests\test_constructor.py	Constructor Initialization	1	__init__ used in test class instead of setUp()	3 SMELLS
	Test Maverick	1	Test class contains only one test method	
	Magic Number Test	6	Magic number 10 in assertion — use a named constant	
tests\test_duplication.py	Magic Number Test	4	Magic number 15 in assertion — use a named constant	2 SMELLS
	Magic Number Test	9	Magic number 15 in assertion — use a named constant	
tests\test_empty.py	Empty Test	2	Test has no body or contains only 'pass'	1 SMELL

File wise smell detection result

```
smell_detection.py X upload.py main.py Settings test_exception.py t
backend > app > services > smell_detection.py > _analyze_function_smells
238 def _analyze_function_smells(func_node: ast.FunctionDef, lines: list):
258
259     body_nodes = func_node.body
260     all_nodes = list(ast.walk(func_node))
261
262     # — ET: Empty Test —————
263     if not body_nodes or (
264         len(body_nodes) == 1 and isinstance(body_nodes[0], ast.Pass)
265     ):
266         smells.append({
267             "type": "Empty Test",
268             "line": func_node.lineno,
269             "message": "Test has no body or contains only 'pass'",
270         })
271         return smells # nothing else to check
272
273     # Also treat a docstring-only body as empty
274     if len(body_nodes) == 1 and isinstance(body_nodes[0], ast.Expr) and isinstance(bo
275         smells.append({
276             "type": "Empty Test",
277             "line": func_node.lineno,
278             "message": "Test contains only a docstring – no assertions",
279         })
280         return smells
281
282     # Collect assertions once – reused by multiple detectors below
283     assertions = [n for n in all_nodes if isinstance(n, ast.Assert)]
284
285     # — AR: Assertion Roulette —————
286     # Multiple assertions with no explanatory message
287     if len(assertions) > 3:
```

Smell detection code

Test Smell Prioritization (Git-based Metrics)

Total Commits: **70** Faulty Commits: **19** Fault Rate: **27.14%**

SMELL TYPE	INSTANCES	CP SCORE	FP SCORE	PRIORITY SCORE	RANKING
Exception Handling	6	0.2162	0.6541	0.4351	#1
Constructor Initialization	1	0.1235	0.6588	0.3911	#2
Test Maverick	1	0.1235	0.6588	0.3911	#3
Magic Number Test	63	0.1127	0.4762	0.2945	#4
Conditional Test Logic	7	0.1806	0.3351	0.2578	#5

Priority score evaluated from Git history analysis

```
index.html  AuthContext.jsx  FRONTEND_GUIDE.txt U  git_metrics.py X  config.py
backend > app > services > git_metrics.py > ...
365 def calculate_spearman_metrics(
389
390     for smell_type, instances in smells_by_type.items():
391         abbr = SMELL_ABBREVIATIONS.get(smell_type, smell_type[:4].upper())
392         smell_files: Set[str] = set(inst['file'] for inst in instances)
393
394         presence: List[float] = [
395             1.0 if any(paths_match(tf, sf) for sf in smell_files) else 0.0
396             for tf in all_test_files
397         ]
398
399         rho_cf, p_cf = _spearman(presence, chg_freq_col)
400         rho_ce, p_ce = _spearman(presence, chg_ext_col)
401         rho_ff, p_ff = _spearman(presence, fault_freq_col)
402         rho_fe, p_fe = _spearman(presence, fault_ext_col)
403
404         cp_score = rho_cf + rho_ce
405         fp_score = rho_ff + rho_fe
406         ps_score = cp_weight * cp_score + (1 - cp_weight) * fp_score
407
408         results[smell_type] = {
409             'smell_type': smell_type,
410             'abbreviation': abbr,
411             'change_frequency_rho': round(rho_cf, 4),
412             'change_extent_rho': round(rho_ce, 4),
413             'fault_frequency_rho': round(rho_ff, 4),
414             'fault_extent_rho': round(rho_fe, 4),
415             'p_values': {
416                 'cf': round(p_cf, 4),
417                 'ce': round(p_ce, 4),
418                 'ff': round(p_ff, 4),
419                 'fe': round(p_fe, 4)
420             }
421         }
```

code snippet of git metrics analysis

Detailed Results per Smell

SMELL	FULL NAME	PS	DDS	NORM. PS	NORM. DDS	QUADRANT	PRIORITY
EH	Exception Handling	0.4351	5.0000	+0.2416	+2.1429	Prudent & Deliberate	HIGH — Refactor Immediately
CI	Constructor Initialization	0.3911	3.0000	+0.1976	+0.1429	Prudent & Deliberate	HIGH — Refactor Immediately
MNT	Magic Number Test	0.2945	4.0000	+0.1010	+1.1429	Prudent & Deliberate	HIGH — Refactor Immediately
CTL	Conditional Test Logic	0.2578	4.5000	+0.0643	+1.6429	Prudent & Deliberate	HIGH — Refactor Immediately
AR	Assertion Roulette	0.2522	3.0000	+0.0587	+0.1429	Prudent & Deliberate	HIGH — Refactor Immediately
ST	Sleepy Test	0.2281	3.0000	+0.0346	+0.1429	Prudent & Deliberate	HIGH — Refactor Immediately
TM	Test Maverick	0.3911	2.0000	+0.1976	-0.8571	Reckless & Deliberate	MODERATE-HIGH — Refactor Soon
DA	Duplicate	0.2522	2.5000	+0.0587	-0.3571	Reckless &	MODERATE-HIGH — Refactor Soon

Final Quadrant Result

```
smell_detection.py  survey_service.py  upload.py  main.py  Settings
backend > app > services > survey_service.py > _classify_quadrant
277 def calculate_quadrant_results(
344     |         "priority": priority,
345     |     })
346
347     # Sort by quadrant importance then by normalizedPS desc
348     _order = {
349         "Prudent & Deliberate": 0,
350         "Reckless & Deliberate": 1,
351         "Prudent & Inadvertent": 2,
352         "Reckless & Inadvertent": 3,
353     }
354     results.sort(key=lambda r: (_order[r["quadrant"]], -r["PS"]))
355
356     return results
357
358
359 def _classify_quadrant(norm_ps: float, norm_dds: float) -> str:
360     if norm_ps >= 0 and norm_dds >= 0:
361         return "Prudent & Deliberate"
362     if norm_ps >= 0 and norm_dds < 0:
363         return "Reckless & Deliberate"
364     if norm_ps < 0 and norm_dds >= 0:
365         return "Prudent & Inadvertent"
366     return "Reckless & Inadvertent"
367
```

Final Quadrant result code

FUTURE WORKS

- 1. Support for More Languages:** Currently only Python projects are supported. Extending to Java (with JUnit), JavaScript (with Jest), or C# would make the tool useful to a much wider audience.
- 2. IDE Plugin:** Right now the tool is a standalone web app. Integrating it as a VS Code or PyCharm plugin would let developers see real-time smell rankings .

CHALLENGES

Challenge 1 — Git Metrics Calculation Pipeline The full ICSME 2025 paper methodology had to be implemented from scratch — parsing git history, building co-change maps, constructing four metric vectors per file, and computing Spearman correlation. Any mistake in any step silently produces wrong rankings with no error or warning.

Challenge 2 — AST-Based Detection of 15 Smells Writing a custom AST parser for all 15 smell types without any external library required deeply understanding how Python represents every code construct.

Challenge 3 — Rolling DDS and Quadrant Recalculation Every new survey submission triggers a full DDS recalculation and quadrant reclassification — where the mean shifts with each response, meaning quadrant assignments can change over time. Managing this correctly across three database collections while preventing duplicate submissions was the most complex part of the system.

EXAMPLE TEST CASE

Test Case ID: T1

Title: Repository Input Processing

Test Scenario:

Submit a GitHub repository URL and upload a local ZIP file through the web interface to verify input validation and processing.

Expected Outcome:

The GitHub repository is cloned and the ZIP file is extracted successfully. The system proceeds to test smell detection without errors.

REFERENCE

Hasan, M. A., & Ahammed, T. (2025, September). Prioritizing test smells: An empirical evaluation of quality metrics and developer perceptions. In Proceedings of the 41st International Conference on Software Maintenance and Evolution (ICSME 2025) – New Ideas and Emerging Results Track, University of Auckland, Auckland, New Zealand.

THANK YOU